Universidad de Cundinamarca

Extensión Chia

Facultad de ingeniería

ingeniería de Sistemas y computación

ingeniería de Software I

Grupo T1

R2-A1-S5 Documento de requerimientos del sistema

Trabajo: API Rest

Brayan Esteban Pinzon Torres

Profesor:

William Alexander Matallana Porras

18/03/2026

Chía Cundinamarca

Tabla de Contenido

R2-A1-S5 1

Introducción ***3***

Objetivo General ***3***

Tecnologías utilizadas ***3***

Arquitectura del proyecto ***4***

Relaciones entre entidades ***4***

Base de datos ***4***

Objetivos 5

Objetivo general ***5***

Objetivos específicos ***5***

Alcance del proyecto 6

Resultados esperados 6

CRUD 1 7

CRUD 2 8

CRUD 3 9

CRUD 4 10

CRUD 5 10

CRUD 6 11

Pruebas swagger 12

Introducción

Objetivo General

Este proyecto tiene como objetivo la creación de una API REST orientada a la administración académica de una universidad. La solución fue desarrollada utilizando el framework Spring Boot, implementando una arquitectura en capas y empleando JPA/Hibernate para la gestión del mapeo entre objetos y la base de datos.

La API facilita la ejecución de operaciones CRUD completas sobre seis entidades clave: Categoria, Libro, Curso, Estudiante, Usuario y PerfilUsuario. Cada una de estas entidades dispone de su respectivo repositorio, capa de servicio (con su implementación) y controlador REST, asegurando así una adecuada organización del código y una clara separación de responsabilidades.

Tecnologías utilizadas

Tabla 1 Tecnologías Utilizadas

Tecnología	Versión	Función
Spring Boot	4.0.3	Framework principal del proyecto
Java	21	Lenguaje de programación
JPA / Hibernate	7.2.4	Mapeo objeto-relacional (ORM)
Supabase	Cloud	Base de datos en la nube
Swagger / OpenAPI	2.5.0	Documentación y prueba de endpoints
Maven	3.9.12	Gestión de dependencias y build
IntelliJ IDEA	2025.3.2	Entorno de desarrollo (IDE)

Arquitectura del proyecto

La aplicación está estructurada bajo un enfoque de arquitectura en capas claramente organizado:

• **Model** — Clases de entidad JPA encargadas de representar las tablas de la base de datos.

• **Repository** — Interfaces que heredan de JpaRepository y permiten la interacción con la base de datos.

• **Service** — Definición de las operaciones de negocio mediante interfaces.

• **ServiceImpl** — Implementaciones de los servicios donde se desarrolla la lógica y validaciones necesarias.

• **Controller** — Componentes REST que exponen los endpoints disponibles de la API.

• **Configuration** — Configuración de Swagger para la generación automática de la documentación.

Relaciones entre entidades

Las entidades del sistema se encuentran vinculadas mediante las siguientes relaciones:

• **Categoria — Libro**: asociación uno a muchos (OneToMany), donde una categoría puede contener varios libros.

• **Libro — Categoria**: relación muchos a uno (ManyToOne), indicando que cada libro pertenece a una única categoría.

• **Estudiante — Curso**: relación muchos a muchos (ManyToMany), gestionada mediante la tabla intermedia *estudiante_curso*.

• **Usuario — PerfilUsuario**: relación uno a uno (OneToOne), donde cada usuario posee un solo perfil asociado.

Base de datos

Spring Boot se encarga de su configuración automática al iniciar

la aplicación, mientras que Hibernate genera las tablas basándose en las anotaciones JPA definidas en las entidades.

Objetivos

Objetivo general

Construir una API REST operativa para la gestión académica universitaria, aplicando buenas prácticas de desarrollo con Spring Boot, utilizando el patrón Service/ServiceImpl y proporcionando documentación clara de los endpoints mediante Swagger para facilitar su uso y validación.

Objetivos específicos

1. Desarrollar el modelo de datos con las seis entidades principales (Categoria, Libro, Curso, Estudiante, Usuario y PerfilUsuario), definiendo sus relaciones mediante anotaciones JPA.
2. Implementar repositorios para cada entidad extendiendo JpaRepository, evitando la necesidad de escribir consultas SQL manuales.
3. Definir interfaces de servicio que incluyan operaciones como listar, guardar, actualizar y eliminar, siguiendo principios de diseño orientado a interfaces.
4. Desarrollar la lógica de negocio en las clases ServiceImpl, incorporando validaciones y manejo de errores con mensajes claros.
5. Crear controladores REST que permitan ejecutar operaciones CRUD, utilizando rutas bajo el esquema /api/{entidad}/{operacion}.
6. Integrar Swagger/OpenAPI para generar documentación automática e interactiva de la API.

7. Configurar la conexión a la base de datos a través del archivo application.properties, usando H2 en memoria para el desarrollo.
8. Validar el funcionamiento de los endpoints mediante pruebas desde Swagger, verificando respuestas HTTP y datos obtenidos.

Alcance del proyecto

El proyecto contempla el desarrollo completo del backend de la API, incluyendo:

• Seis modelos de datos con relaciones (OneToMany, ManyToOne, ManyToMany y

OneToOne).

• Seis repositorios para acceso a datos mediante JpaRepository.

• Seis interfaces de servicio junto con sus implementaciones y validaciones.

• Seis controladores REST con cuatro endpoints cada uno (listar, guardar, actualizar y eliminar).

• Documentación interactiva generada con Swagger UI disponible en /swagger-ui/index.html.

• Base de datos H2 en memoria para pruebas y desarrollo.

Resultados esperados

Al concluir el proyecto, se espera disponer de una API REST completamente funcional que permita:

• Administrar categorías y los libros asociados a cada una.

• Gestionar cursos académicos y la asignación de estudiantes.

• Controlar los usuarios del sistema junto con su información de perfil.

• Probar de forma interactiva todos los endpoints mediante Swagger UI.

• Observar en consola las consultas SQL generadas automáticamente por Hibernate.

CRUD 1

Tabla 2 CRUD 1

Operación	Consulta SQL	Método JPA	Descripción
CREATE	INSERT INTO categorias (...) VALUES (...)	save(entidad)	Inserta un nuevo registro
READ ALL	SELECT * FROM categorias	findAll()	Lista todos los registros
READ BY ID	SELECT * FROM categorias WHERE id = ?	findById(id)	Busca por clave primaria
UPDATE	UPDATE categorias SET ... WHERE id = ?	save(entidad)	Actualiza registro existente
DELETE	DELETE FROM categorias WHERE id = ?	deleteById(id)	Elimina por clave primaria

CRUD 2

Tabla 3 Crud 2

Operación	Consulta SQL	Método JPA	Descripción
CREATE	INSERT INTO libros (...) VALUES (...)	save(entidad)	Inserta un nuevo registro
READ ALL	SELECT * FROM libros	findAll()	Lista todos los registros

Operación	Consulta SQL	Método JPA	Descripción
READ BY ID	SELECT * FROM libros WHERE id = ?	findById(id)	Busca por clave primaria
UPDATE	UPDATE libros SET ... WHERE id = ?	save(entidad)	Actualiza registro existente
DELETE	DELETE FROM libros WHERE id = ?	deleteById(id)	Elimina por clave primaria

CRUD 3

Tabla 4 Crud 3

Operación	Consulta SQL	Método JPA	Descripción
CREATE	INSERT INTO cursos (...) VALUES (...)	save(entidad)	Inserta un nuevo registro
READ ALL	SELECT * FROM cursos	findAll()	Lista todos los registros
READ BY ID	SELECT * FROM cursos WHERE id = ?	findById(id)	Busca por clave primaria
UPDATE	UPDATE cursos SET ... WHERE id = ?	save(entidad)	Actualiza registro existente

Operación	Consulta SQL	Método JPA	Descripción
DELETE	DELETE FROM cursos WHERE id = ?	deleteById(id)	Elimina por clave primaria

Tabla 5 CRUD 3

CRUD 4

Operación	Consulta SQL	Método JPA	Descripción
CREATE	INSERT INTO estudiantes (...) VALUES (...)	save(entidad)	Inserta un nuevo registro
READ ALL	SELECT * FROM estudiantes	findAll()	Lista todos los registros
READ BY ID	SELECT * FROM estudiantes WHERE id = ?	findById(id)	Busca por clave primaria
UPDATE	UPDATE estudiantes SET ... WHERE id = ?	save(entidad)	Actualiza registro existente
DELETE	DELETE FROM estudiantes WHERE id = ?	deleteById(id)	Elimina por clave primaria

Tabla 6 CRUD 4

CRUD 5

Operación	Consulta SQL	Método JPA	Descripción
CREATE	INSERT INTO usuarios (...) VALUES (...)	save(entidad)	Inserta un nuevo registro
READ ALL	SELECT * FROM usuarios	findAll()	Lista todos los registros
READ BY ID	SELECT * FROM usuarios WHERE id = ?	findById(id)	Busca por clave primaria
UPDATE	UPDATE usuarios SET ... WHERE id = ?	save(entidad)	Actualiza registro existente
DELETE	DELETE FROM usuarios WHERE id = ?	deleteById(id)	Elimina por clave primaria

CRUD 6

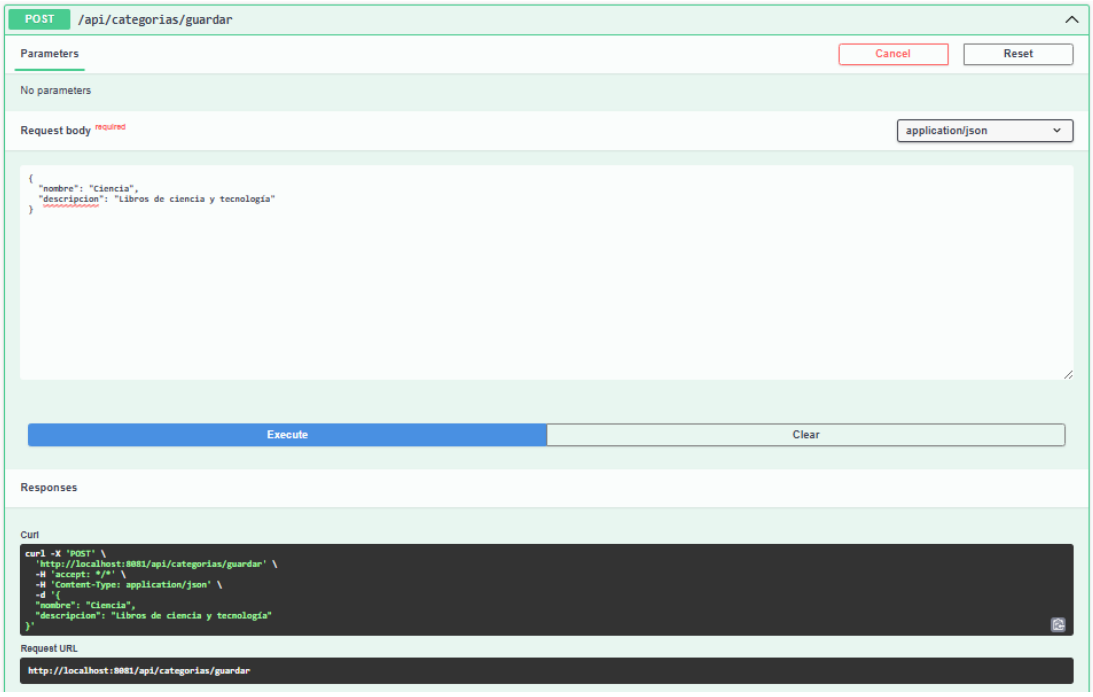
Operación	Consulta SQL	Método JPA	Descripción
CREATE	INSERT INTO perfiles_usuario (...) VALUES (...)	save(entidad)	Inserta un nuevo registro
READ ALL	SELECT * FROM perfiles_usuario	findAll()	Lista todos los registros

Operación	Consulta SQL	Método JPA	Descripción
READ BY ID	SELECT * FROM perfiles_usuario WHERE id = ?	findById(id)	Busca por clave primaria
UPDATE	UPDATE perfiles_usuario SET ... WHERE id = ?	save(entidad)	Actualiza registro existente
DELETE	DELETE FROM perfiles_usuario WHERE id = ?	deleteById(id)	Elimina por clave primaria

Tabla 7 CRUD 6

Pruebas swagger

- POST /api/categorías/guardar



Server response

Code	Details
201 Undocumented	Response body

```
{
  "id": 1,
  "nombre": "Ciencia",
  "descripcion": "Libros de ciencia y tecnología",
  "libros": null
}
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,18 Mar 2026 11:55:33 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

Media type

/

Controls Accept header.

Example Value | Schema

```
{
  "id": 0,
  "nombre": "string",
  "descripcion": "string",
  "libros": [
    {
      "id": 0,
      "titulo": "string",
      "autor": "string",
      "aniopublicacion": 0,
      "categoria": "string"
    }
  ]
}
```

1. GET /api/categorías/listar

GET /api/categorias/listar

Parameters

Cancel

No parameters

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8081/api/categorias/listar' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8081/api/categorias/listar
```

Server response

Code	Details
200	Response body

```
[
  {
    "id": 1,
    "nombre": "Ciencia",
    "descripcion": "Libros de ciencia y tecnología",
    "libros": []
  },
  {
    "id": 2,
    "nombre": "Historia",
    "descripcion": "Libros sobre eventos y personajes históricos",
    "libros": []
  }
]
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,18 Mar 2026 12:06:10 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

Media type

/

Controls Accept header.

Example Value | Schema

```
[
  {
    "id": 0,
    "nombre": "string",
    "descripcion": "string",
    "libros": [
      {
        "id": 0,
        "titulo": "string",
        "autor": "string",
        "aniopublicacion": 0,
        "categoria": "string"
      }
    ]
  }
]
```

- PUT /api/categorias/actualizar/{id}

PUT /api/categorias/actualizar/{id}

Parameters

Cancel Reset

Name Description

id * required
integer($int64)
(path)
1

Request body required

application/json

```
{
  "nombre": "Ciencia Actualizada",
  "descripcion": "Descripción actualizada de ciencia"
}
```

Execute Clear

Responses

Curl

```
curl -X 'PUT' \
  'http://localhost:8081/api/categorias/actualizar/1' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "nombre": "Ciencia Actualizada",
  "descripcion": "Descripción actualizada de ciencia"
}'
```

Request URL

```
http://localhost:8081/api/categorias/actualizar/1
```

Server response

Code Details

200

Response body

```
{
  "id": 1,
  "nombre": "Ciencia Actualizada",
  "descripcion": "Descripción actualizada de ciencia",
  "libros": [
    {
      "id": 1,
      "titulo": "El Universo Actualizado",
      "autor": "Stephen Hawking",
      "aniopublicacion": 2005
    }
  ]
}
```

Download

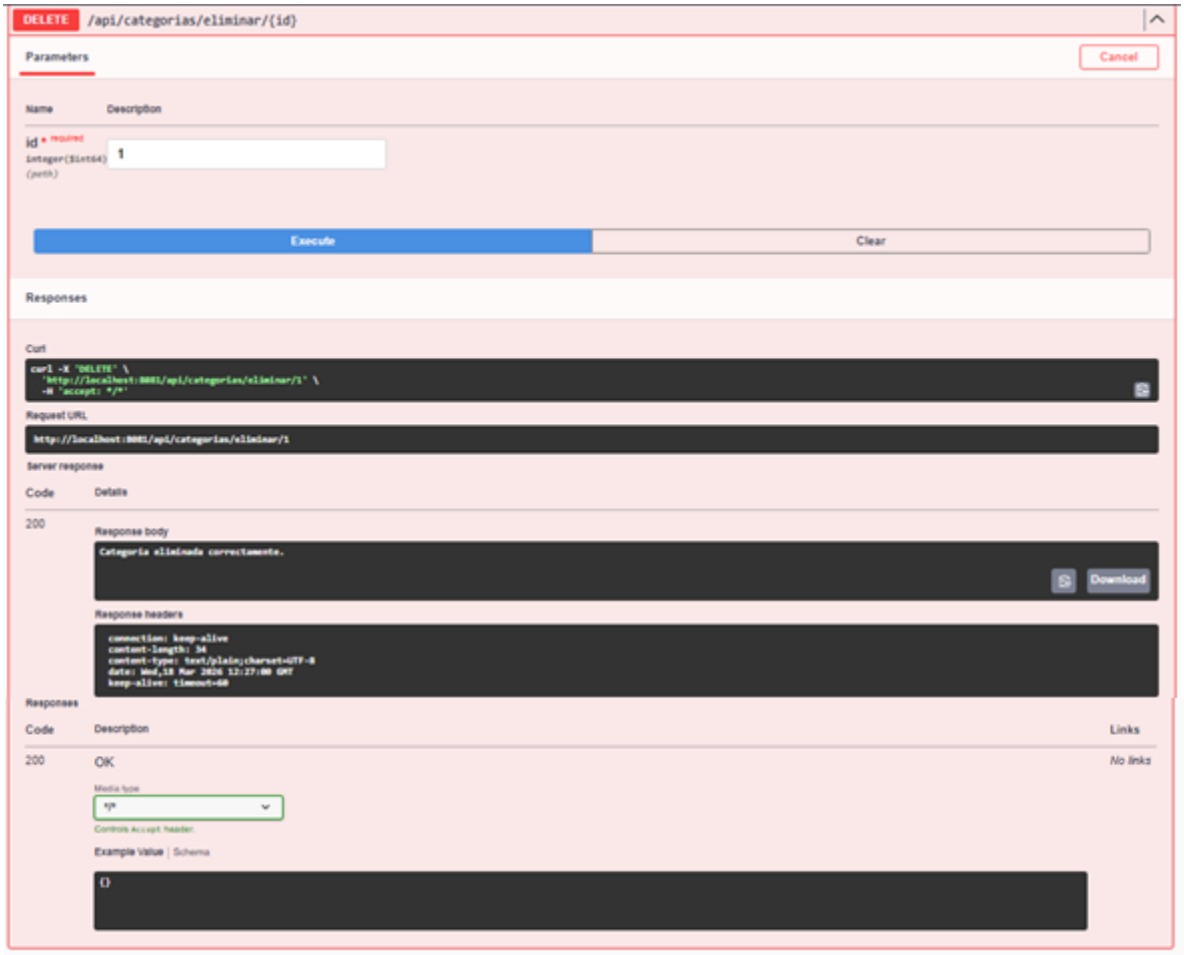
Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,18 Mar 2026 12:23:37 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

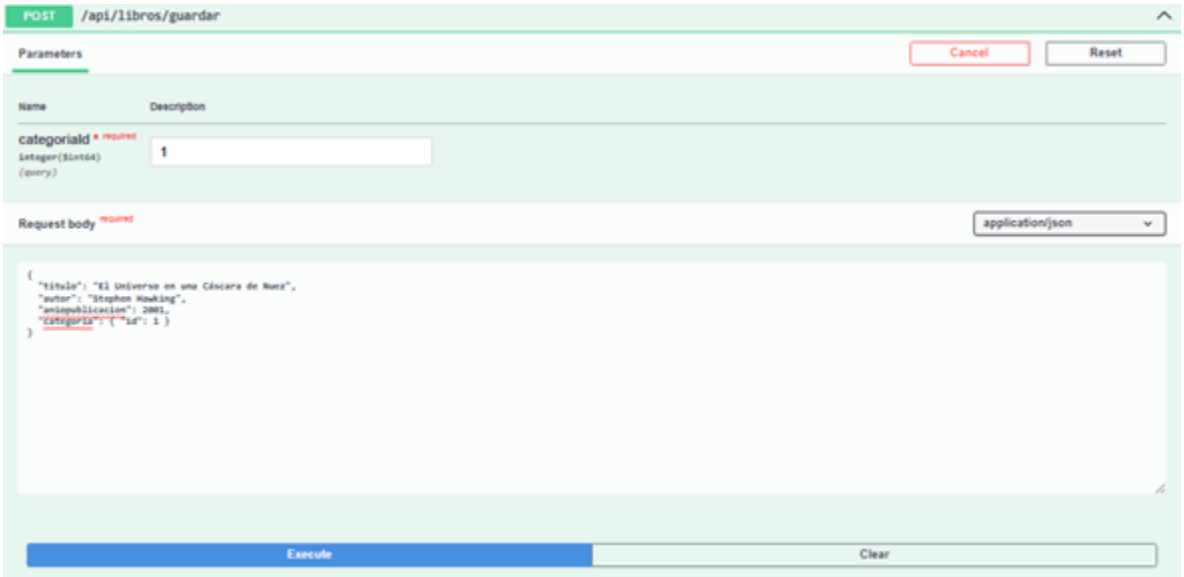
Responses

Code	Description	Links
200	OK Media type */* Controls Accept header. Example Value | Schema	No links

- DELETE /api/categorias/eliminar/{id}



- POST /api/libros/guardar



Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8081/api/libros/guardar?categoriaId=1' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "titulo": "El Universo en una Cáscara de Nuez",
  "autor": "Stephen Hawking",
  "aniopublicacion": 2001,
  "categoria": { "id": 1 }
}'
```

Request URL

```
http://localhost:8081/api/libros/guardar?categoriaId=1
```

Server response

Code	Details
201 *Undocumented*	Response body

```
{
  "id": 1,
  "titulo": "El Universo en una Cáscara de Nuez",
  "autor": "Stephen Hawking",
  "aniopublicacion": 2001
}
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,18 Mar 2026 12:13:30 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	*No links*

Media type

/

Controls Accept header.

Example Value | Schema

GET /api/libros/listar

Parameters

Cancel

No parameters

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8081/api/libros/listar' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8081/api/libros/listar
```

Server response

Code	Details
200	Response body

```
[
  {
    "id": 1,
    "titulo": "El Universo en una Cáscara de Nuez",
    "autor": "Stephen Hawking",
    "aniopublicacion": 2001
  }
]
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,18 Mar 2026 12:15:40 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	*No links*

Media type

/

Controls Accept header.

Example Value | Schema

```
[
  {
    "id": 0,
    "titulo": "string",
    "autor": "string",
    "aniopublicacion": 0,
    "categoria": "string"
  }
]
```

- PUT /api/libros/actualizar/{id}

PUT /api/libros/actualizar/{id}

Parameters

Cancel Reset

Name	Description
id * required integer($int64) (path)	1
categoriaId integer($int64) (query)	1

Request body required

application/json

```
{
  "titulo": "El Universo Actualizado",
  "autor": "Stephen Hawking",
  "aniopublicacion": 2005,
  "categoria": { "id": 1 }
}
```

Execute Clear

Responses

Curl

```
curl -X 'PUT' \
  'http://localhost:8081/api/libros/actualizar/1?categoriaId=1' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "titulo": "El Universo Actualizado",
  "autor": "Stephen Hawking",
  "aniopublicacion": 2005,
  "categoria": { "id": 1 }
}'
```

Request URL

```
http://localhost:8081/api/libros/actualizar/1?categoriaId=1
```

Server response

Code	Details
200	Response body

```
{
  "id": 1,
  "titulo": "El Universo Actualizado",
  "autor": "Stephen Hawking",
  "aniopublicacion": 2005
}
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,18 Mar 2026 12:18:38 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK Media type */* Controls Accept header. Example Value | Schema {}	No links

- POST /api/cursos/guardar

POST /api/cursos/guardar

Parameters

Cancel Reset

No parameters

Request body required

application/json

```
{
  "nombres": "Matemáticas",
  "creditos": 3
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8081/api/cursos/guardar' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "nombres": "Matemáticas",
  "creditos": 3
}'
```

Request URL

```
http://localhost:8081/api/cursos/guardar
```

Server response

Code	Details
201 Undocumented	Response body

```
{
  "id": 1,
  "nombres": "Matemáticas",
  "creditos": 3,
  "estudiantes": null
}
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,18 Mar 2026 12:34:52 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

Media type

/

Controls Accept header.

Example Value | Schema

```
{
  "id": 0,
  "nombres": "string",
  "creditos": 0,
  "estudiantes": [
    {
      "id": 0,
      "nombre": "string",
      "correo": "string",
      "estudiantes": [
        "string"
      ]
    }
  ]
}
```

- GET /api/cursos/listar

GET /api/cursos/listar

Parameters

Cancel

No parameters

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8081/api/cursos/listar' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8081/api/cursos/listar
```

Server response

Code	Details
200	Response body

```
[
  {
    "id": 1,
    "nombres": "Matemáticas",
    "creditos": 3,
    "estudiantes": []
  }
]
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,18 Mar 2026 12:37:44 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

Media type

/

Controls Accept header.

Example Value | Schema

```
[
  {
    "id": 0,
    "nombres": "string",
    "creditos": 0,
    "estudiantes": [
      {
        "id": 0,
        "nombre": "string",
        "correo": "string",
        "estudiantes": [
          "string"
        ]
      }
    ]
  }
]
```

- PUT /api/cursos/actualizar/{id}

PUT /api/categorias/actualizar/{id}

Parameters

Cancel | Reset

Name	Description
id * required integer($int64) (path)	1

Request body required

application/json

```
{
  "nombre": "Ciencia Actualizada",
  "descripcion": "Descripción actualizada de ciencia"
}
```

Execute | Clear

Curl

```
curl -X 'PUT' \
  'http://localhost:8081/api/categorias/actualizar/1' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "nombre": "Ciencia Actualizada",
  "descripcion": "Descripción actualizada de ciencia"
}'
```

Request URL

```
http://localhost:8081/api/categorias/actualizar/1
```

Server response

Code	Details
400 *Undocumented*	Error: response status is 400

Response body

```
Categoria no encontrada con id: 1
```

Download

Response headers

```
connection: close
content-length: 33
content-type: text/plain;charset=UTF-8
date: Wed,18 Mar 2026 12:39:39 GMT
```

Responses

Code	Description	Links
200	OK	*No links*

Media type

/

Controls Accept header.

Example Value | Schema

```
{}
```

- DELETE /api/cursos/eliminar/{id}

DELETE /api/cursos/eliminar/{id}

Parameters

Cancel

Name	Description
id * required integer($int64) *(path)*	1

Execute | Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://localhost:8081/api/cursos/eliminar/1' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8081/api/cursos/eliminar/1
```

Server response

Code	Details
200	

Response body

```
Curso eliminado correctamente.
```

Download

Response headers

```
connection: keep-alive
content-length: 30
content-type: text/plain;charset=UTF-8
date: Wed,18 Mar 2026 12:47:19 GMT
keep-alive: timeout=60
```

Responses

Code	Description	Links
200	OK	*No links*

Media type

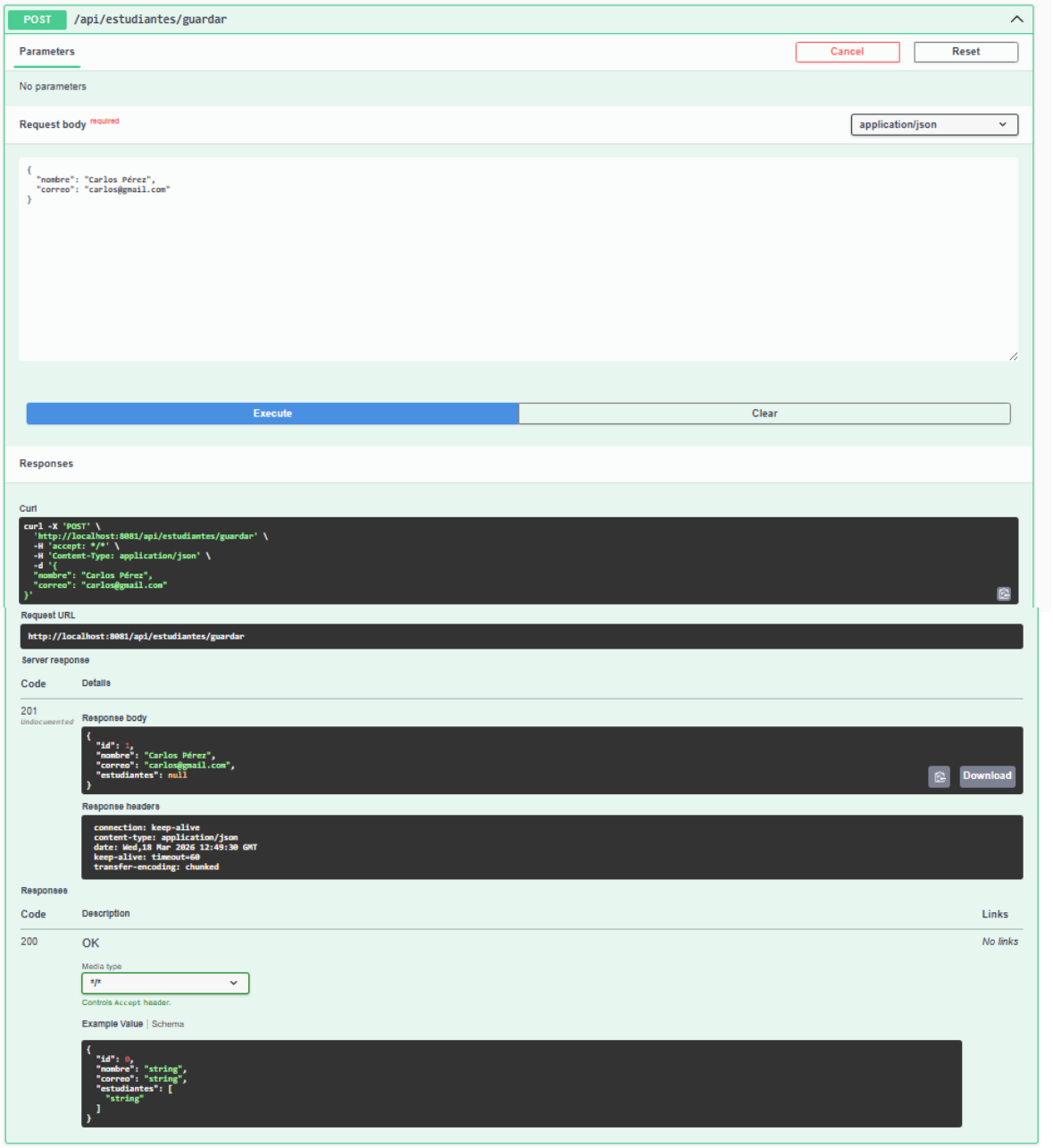
/

Controls Accept header.

Example Value | Schema

```
{}
```

- POST /api/estudiantes/guardar



- GET /api/estudiantes/listar

GET /api/estudiantes/listar

Parameters

Cancel

No parameters

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8081/api/estudiantes/listar' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8081/api/estudiantes/listar
```

Server response

Code | Details

200

Response body

```
[
  {
    "id": 1,
    "nombre": "Carlos Pérez",
    "correo": "carlos@gmail.com",
    "estudiantes": []
  }
]
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,18 Mar 2026 12:52:49 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

Media type

/

Controls Accept header.

Example Value | Schema

```
[
  {
    "id": 0,
    "nombre": "string",
    "correo": "string",
    "estudiantes": [
      "string"
    ]
  }
]
```

- PUT /api/estudiantes/actualizar/{id}

PUT /api/estudiantes/actualizar/{id}

Parameters

Cancel | Reset

Name	Description
id * required integer($int64) (path)	1

Request body required

application/json

```
{
  "nombre": "Carlos Pérez Actualizado",
  "correo": "carlos.nuevo@gmail.com"
}
```

Execute | Clear

Responses

Curl

```
curl -X 'PUT' \
  'http://localhost:8081/api/estudiantes/actualizar/1' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "nombre": "Carlos Pérez Actualizado",
  "correo": "carlos.nuevo@gmail.com"
}'
```

Request URL

```
http://localhost:8081/api/estudiantes/actualizar/1
```

Server response

Code	Details
200	Response body

```
{
  "id": 1,
  "nombre": "Carlos Pérez Actualizado",
  "correo": "carlos.nuevo@gmail.com",
  "estudiantes": []
}
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,18 Mar 2026 12:57:00 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK Media type: */* Controls Accept header. Example Value | Schema {}	No links

- DELETE /api/estudiantes/eliminar/{id}

DELETE /api/estudiantes/eliminar/{id}

Parameters

Cancel

Name	Description
id * required integer($int64) (path)	1

Execute | Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://localhost:8081/api/estudiantes/eliminar/1' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8081/api/estudiantes/eliminar/1
```

Server response

Code	Details
200	Response body

```
Estudiante eliminado correctamente.
```

Download

Response headers

```
connection: keep-alive
content-length: 35
content-type: text/plain;charset=UTF-8
date: Wed,18 Mar 2026 12:59:29 GMT
keep-alive: timeout=60
```

Responses

Code	Description	Links
200	OK Media type: */* Controls Accept header. Example Value | Schema {}	No links

- POST /api/usuarios/guardar

POST /api/usuarios/guardar

Parameters

Cancel Reset

No parameters

Request body required

application/json

```
{
  "nombre": "Manuel García",
  "correo": "manuel@gmail.com"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8081/api/usuarios/guardar' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "nombre": "Manuel García",
  "correo": "manuel@gmail.com"
}'
```

Request URL

http://localhost:8081/api/usuarios/guardar

Server response

Code	Details

201 Undocumented	Response body
	{
	"id": 1,
	"nombre": "Manuel García",
	"correo": "manuel@gmail.com",
	"perfil": null
	}
	Download
	Response headers
	connection: keep-alive
	content-type: application/json
	date: Wed,18 Mar 2026 13:05:39 GMT
	keep-alive: timeout=60
	transfer-encoding: chunked

Responses

Code	Description	Links

200	OK
	Media type: */*
	Controls Accept header.
	Example Value | Schema
	{
	"id": 0,
	"nombre": "string",
	"correo": "string",
	"perfil": {
	"id": 0,
	"documento": "string",
	"telefono": "string"
	}
	}
		No links

- GET /api/usuarios/listar

GET /api/usuarios/listar

Parameters

Cancel

No parameters

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8081/api/usuarios/listar' \
  -H 'accept: */*'
```

Request URL

http://localhost:8081/api/usuarios/listar

Server response

Code	Details
200	Response body

```
[
  {
    "id": 1,
    "nombre": "Manuel García",
    "correo": "manuel@gmail.com",
    "perfil": null
  }
]
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,18 Mar 2026 13:13:06 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

Media type

/

Controls Accept header.

Example Value | Schema

```
[
  {
    "id": 0,
    "nombre": "string",
    "correo": "string",
    "perfil": {
      "id": 0,
      "documento": "string",
      "telefono": "string"
    }
  }
]
```

- PUT /api/usuarios/actualizar/{id}

PUT /api/usuarios/actualizar/{id}

Parameters

Cancel | Reset

Name	Description
id * required integer($int64) (path)	1

Request body required

application/json

```
{
  "nombre": "Manuel García Actualizado",
  "correo": "manuel.nuevo@gmail.com"
}
```

Execute | Clear

Curl

```
curl -X 'PUT' \
  'http://localhost:8081/api/usuarios/actualizar/1' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "nombre": "Manuel García Actualizado",
  "correo": "manuel.nuevo@gmail.com"
}'
```

Request URL

```
http://localhost:8081/api/usuarios/actualizar/1
```

Server response

Code	Details
200	Response body

```
{
  "id": 1,
  "nombre": "Manuel García Actualizado",
  "correo": "manuel.nuevo@gmail.com",
  "perfil": null
}
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed,18 Mar 2026 13:16:15 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

Media type

/

Controls Accept header.

Example Value | Schema

```
{}
```

- DELETE /api/usuarios/eliminar/{id}

DELETE /api/usuarios/eliminar/{id}

Parameters

Cancel

Name	Description
id * required integer($int64) (path)	1

Execute | Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://localhost:8081/api/usuarios/eliminar/1' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8081/api/usuarios/eliminar/1
```

Server response

Code	Details
400 Undocumented	Error: response status is 400

Response body

```
No es posible eliminar. Usuario no encontrado con id: 1
```

Download

Response headers

```
connection: close
content-length: 55
content-type: text/plain;charset=UTF-8
date: Wed,18 Mar 2026 13:18:01 GMT
```

Responses

Code	Description	Links
200	OK	No links

Media type

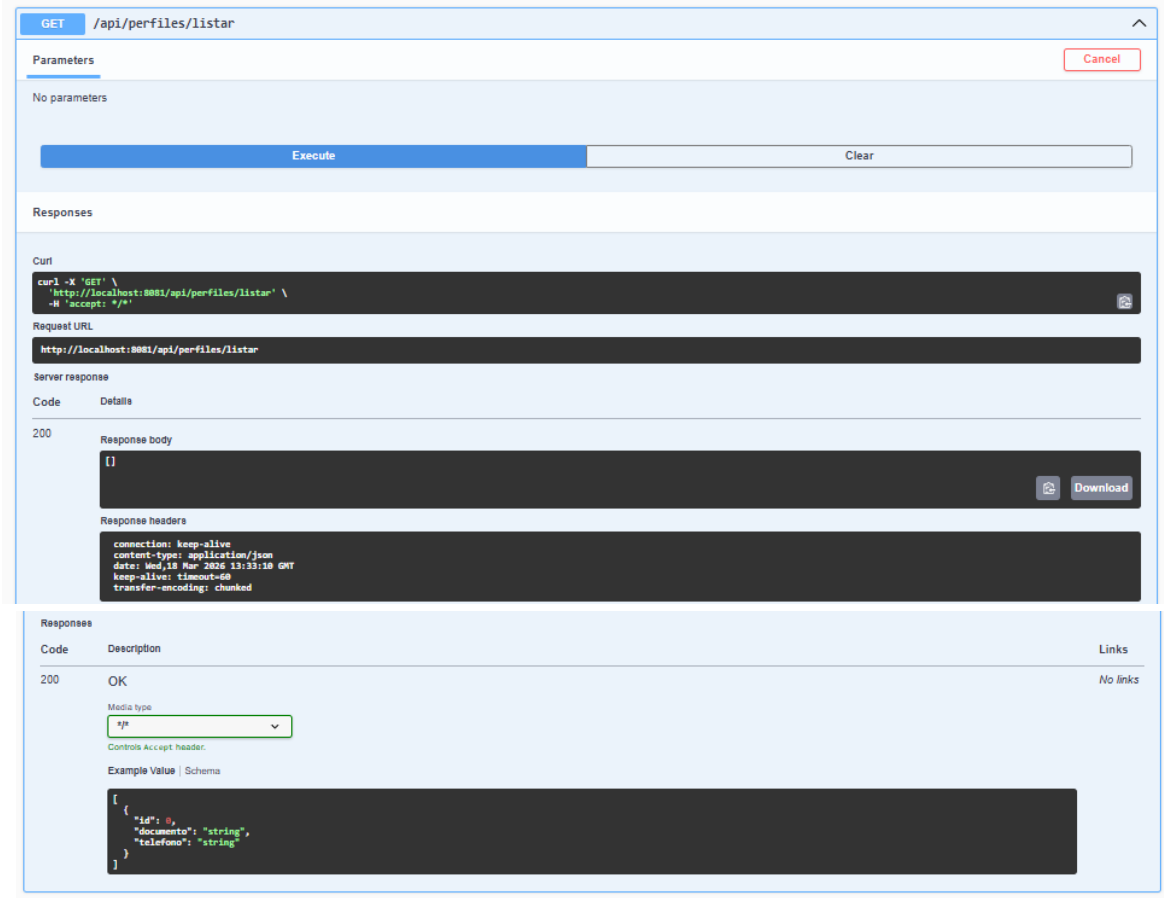
/

Controls Accept header.

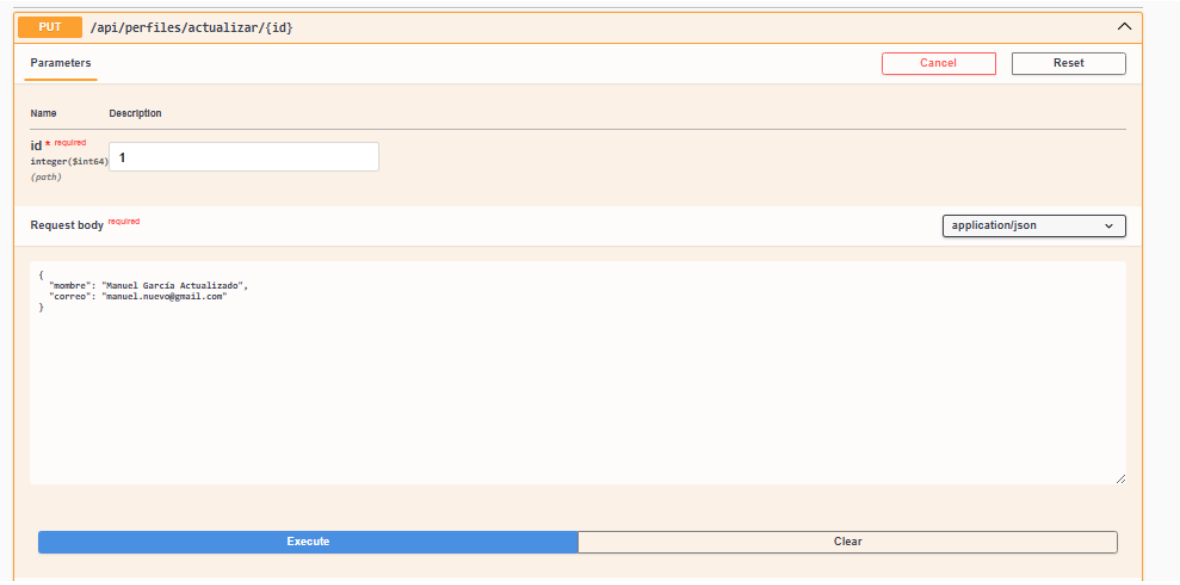
Example Value | Schema

```
{}
```

- GET /api/perfiles/listar



- PUT /api/perfiles/actualizar/{id}



Responses

Curl

```
curl -X 'PUT' \
  'http://localhost:8081/api/perfiles/actualizar/1' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "nombre": "Manuel García Actualizado",
  "correo": "manuel.nuevo@gmail.com"
}'
```

Request URL

```
http://localhost:8081/api/perfiles/actualizar/1
```

Server response

Code	Details
400 *undocumented*	Error: response status is 400

Response body

```
Perfil no encontrado con id: 1
```

Download

Response headers

```
connection: close
content-length: 30
content-type: text/plain;charset=UTF-8
date: Wed,18 Mar 2026 13:35:21 GMT
```

Responses

Code	Description	Links
200	OK	*No links*

Media type

/

Controls Accept header.

Example Value | Schema

```
{}
```